



Universidade Federal de Campina Grande
Centro de Competência EMBRAPPII VIRTUS em Hardware Inteligente
para a Indústria

Guia de Referência Definitivo

Arquitetura e Operação do Timer EF_TMR32 (Wishbone)

Orientadores: Professor Dr. Marcos Ricardo Alcântara Morais
Professor Dr. Gutemberg Gonçalves dos Santos Júnior

Campina Grande — PB

13 de julho de 2026

Sumário

1	Introdução Arquitetural	1
2	Pinagem e Mapa de Sinais	2
2.1	Interface de Barramento Wishbone (Slave)	2
2.2	Sinais de Interface Funcional (I/O)	3
3	Teoria de Operação do Núcleo	4
3.1	A Hierarquia de Clock e o Prescaler	4
3.2	Modos de Contagem e Comportamento de Transição	4
4	Mapa de Registradores e Funcionalidade de Bits	6
4.1	Tabela de Registradores	6
4.2	Detalhamento de Bits	7
4.2.1	Registradores TMR, RELOAD, PR, CMPX, CMPY	7
4.2.2	Registrador CTRL (0x0014)	7
4.2.3	Registrador CFG (0x0018)	8
4.2.4	Registradores PWM0CFG (0x1C) e PWM1CFG (0x20)	9
4.2.5	Registrador PWMDT (0x0024)	11
4.2.6	Registrador PWMFC (0x0028)	11
4.2.7	Registradores de Interrupção (IM, RIS, MIS, IC)	12
4.2.8	Registrador GCLK (0xFF10)	12
5	Geração de PWM Baseada em Eventos: Detalhes da Lógica	14
6	Interface de Controle Wishbone B4: Protocolo e Integração	15
6.1	Protocolo de Acesso Detalhado	15
7	Guia de Programação (Exemplos Práticos)	16
8	Conclusão e Melhores Práticas	18
	Referências	19

Tabela 1: Revisão Histórica da Documentação

Versão	Data	Autor(es)	Descrição das Alterações
1.0	13 de julho de 2026	Manus AI	Criação do documento sobre a arquitetura do Timer EF_TMR32.

Resumo

Este manual transcende uma simples folha de dados, oferecendo uma análise arquitetural profunda do IP EF_TMR32. Exploramos a engenharia por trás da geração de PWM baseada em eventos, a hierarquia de controle do barramento Wishbone B4 e as estratégias de implementação para sistemas de controle de tempo real e eletrônica de potência. As informações foram consolidadas a partir do código RTL, do datasheet oficial e da documentação do barramento Wishbone, resolvendo ambiguidades e fornecendo uma visão técnica exhaustiva.

1 Introdução Arquitetural

O EF_TMR32 não é meramente um contador de pulsos; ele é um **processador de sinais de temporização autônomo** [1]. Projetado para desonerar a CPU principal, ele gerencia tarefas complexas de modulação e temporização diretamente no hardware. Sua arquitetura modular e a interface Wishbone padronizada facilitam a integração em sistemas baseados em microcontroladores ou FPGAs.

A arquitetura é centrada em um núcleo de contagem de 32 bits alimentado por um *prescaler* regressivo. A grande inovação deste IP reside na sua **Matriz de Decisão de Eventos**, que permite que mudanças no estado dos pinos de saída sejam disparadas por coincidências matemáticas entre o contador e registradores de comparação, sem a necessidade de interrupções de software para cada transição de borda.

2 Pinagem e Mapa de Sinais

A interface física do módulo é projetada para ser robusta e fácil de rotear. As informações a seguir foram consolidadas do datasheet [2] e do código RTL [3].

2.1 Interface de Barramento Wishbone (Slave)

O barramento segue a especificação Wishbone B4 em modo escravo [4].

Sinal	Direção	Descrição Detalhada
clk_i	Input	Clock do Sistema: Todas as operações internas são sincronizadas na borda de subida deste sinal.
rst_i	Input	Reset Síncrono: Quando alto, reinicia todos os registradores para seus valores padrão. Deve ser mantido alto por pelo menos um ciclo de clock.
adr_i [31:0]	Input	Barramento de Endereço: Seleciona o registrador alvo. Embora o barramento seja de 32 bits, apenas os bits inferiores (16 bits) são decodificados para os offsets internos.
dat_i [31:0]	Input	Dados de Escrita: Dados enviados pelo mestre para serem armazenados nos registradores.
dat_o [31:0]	Output	Dados de Leitura: Conteúdo do registrador selecionado enviado de volta ao mestre.
sel_i [3:0]	Input	Seleção de Byte: Permite operações em partes da palavra de 32 bits (essencial para compatibilidade com diferentes larguras de dados).
cyc_i	Input	Cycle: Indica que um mestre deseja realizar um ou mais ciclos de barramento.

Sinal	Direção	Descrição Detalhada
stb_i	Input	Strobe: Indica que o endereço e os sinais de controle atuais são válidos.
we_i	Input	Write Enable: Define se a operação é de Escrita (1) ou Leitura (0).
ack_o	Output	Acknowledge: Sinaliza ao mestre que a operação foi concluída com sucesso.

2.2 Sinais de Interface Funcional (I/O)

Sinal	Direção	Descrição Detalhada
IRQ	Output	Interrupt Request: Sinal de nível alto que indica um evento de interrupção (Timeout, Match X ou Match Y). Permanece alto até ser limpo via registrador IC [3].
pwm0 / pwm1	Output	Saídas PWM: Sinais de onda quadrada gerados pela lógica de comparação. Podem ser invertidos ou atrasados pelo <i>dead time</i> . São desabilitados em caso de falha [2].
pwm_fault	Input	Entrada de Falha: Um sinal assíncrono que, quando ativado, força as saídas PWM para o estado inativo instantaneamente via hardware, ignorando o controle de software. É um mecanismo de segurança crítico [2].

3 Teoria de Operação do Núcleo

3.1 A Hierarquia de Clock e o Prescaler

O fluxo de tempo no IP começa no sinal `clk_i`. Antes de atingir o contador principal, o clock passa por dois estágios:

1. **Clock Gating (GCLK)**: Uma porta lógica física controlada pelo bit 0 do registrador GCLK. Quando desabilitado ($GCLK[0]=0$), o clock `clk_g` para o núcleo do timer é cortado, economizando energia. O barramento Wishbone, no entanto, continua operando com `clk_i`, permitindo acesso aos registradores mesmo com o timer inativo [3].
2. **Prescaler (PR)**: Um contador regressivo de 32 bits (configurável via parâmetro PRW) que atua como um divisor de frequência. Se o registrador PR contiver o valor N , o contador principal só será incrementado/decrementado a cada $N + 1$ ciclos de `clk_g`. A frequência efetiva de contagem é $F_{clk_g}/(PR + 1)$ [2]. Isso permite que um clock de sistema de 100MHz, por exemplo, controle eventos que ocorrem em intervalos de segundos.

3.2 Modos de Contagem e Comportamento de Transição

O comportamento do contador (TMR) nos limites é definido pelo registrador CFG [2]:

- **Modo Up** ($CFG[1:0]=10$): O contador inicia em 0 e sobe até igualar RELOAD. Ao atingir RELOAD, ele gera um sinal de *Timeout*. Se o modo for **Periódico** ($CFG[2]=1$), ele reseta para 0 no ciclo seguinte e reinicia. Se for **One-shot** ($CFG[2]=0$), ele para em RELOAD e desabilita o bit TE (Timer Enable) no registrador CTRL.
- **Modo Down** ($CFG[1:0]=01$): O contador é carregado com o valor de RELOAD e decresce até 0. Ao atingir 0, gera o *Timeout*. O comportamento Periódico/One-shot é análogo ao modo Up.
- **Modo Up-Down** ($CFG[1:0]=11$): Este modo é essencial para PWM de fase correta. O contador sobe de $0 \rightarrow RELOAD$ e, ao atingir o topo, inverte a direção

automaticamente para $RELOAD \rightarrow 0$. Ao atingir 0, inverte novamente para Up. Isso cria uma forma de onda triangular, ideal para minimizar harmônicos em controle de motores e aplicações de áudio.

4 Mapa de Registradores e Funcionalidade de Bits

O controle do IP é realizado através de 16 registradores principais, acessíveis via barramento Wishbone. O acesso é de 32 bits, mas a largura efetiva de alguns registradores pode ser menor. Os offsets são baseados no datasheet [2] e verificados com o RTL [3].

4.1 Tabela de Registradores

Nome	Offset	Reset	Acesso	Descrição
TMR	0x0000	0x0	R	Valor atual do contador do Timer. Leitura de 32 bits.
RELOAD	0x0004	0x0	RW	Valor de carga/limite do contador. Escrita/Leitura de 32 bits.
PR	0x0008	0x0	RW	Valor do Prescaler (Divisor de frequência). Escrita/Leitura de 32 bits (PRW bits efetivos).
CMPX	0x000C	0x0	RW	Registrador de Comparação X (para PWM). Escrita/Leitura de 32 bits.
CMPY	0x0010	0x0	RW	Registrador de Comparação Y (para PWM). Escrita/Leitura de 32 bits.
CTRL	0x0014	0x0	RW	Controle de operação do Timer e PWM. Escrita/Leitura de 7 bits efetivos.
CFG	0x0018	0x0	RW	Configuração de modo e direção. Escrita/Leitura de 3 bits efetivos.
PWM0CFG	0x001C	0x0	RW	Configuração de eventos para PWM 0. Escrita/Leitura de 12 bits efetivos.
PWM1CFG	0x0020	0x0	RW	Configuração de eventos para PWM 1. Escrita/Leitura de 12 bits efetivos.
PWMDT	0x0024	0x0	RW	Configuração do Dead Time. Escrita/-Leitura de 8 bits efetivos.

Nome	Offset	Reset	Acesso	Descrição
PWMFC	0x0028	0x0	W	Limpeza de falha do PWM. Escrita de 16 bits efetivos.
IM	0xFF00	0x0	RW	Máscara de Interrupção. Escrita/Leitura de 3 bits efetivos.
MIS	0xFF04	0x0	R	Status de Interrupção Mascarada. Leitura de 3 bits efetivos.
RIS	0xFF08	0x0	R	Status de Interrupção Bruta (Raw). Leitura de 3 bits efetivos.
IC	0xFF0C	0x0	W	Limpeza de Interrupção. Escrita de 3 bits efetivos.
GCLK	0xFF10	0x0	RW	Habilitação de Clock Gating. Escrita/-Leitura de 1 bit efetivo.

4.2 Detalhamento de Bits

4.2.1 Registradores TMR, RELOAD, PR, CMPX, CMPY

Estes registradores de 32 bits controlam os valores fundamentais de contagem e comparação do timer. Suas estruturas são diretas, com o valor ocupando a totalidade dos 32 bits, conforme ilustrado na Figura 4.1.

4.2.2 Registrador CTRL (0x0014)

Este registrador de 7 bits controla as operações básicas do timer e PWM [2]. A Figura 4.2 detalha seu mapeamento de bits.

- **Bit 0 (TE - Timer Enable):** 1: Habilita a contagem do timer. 0: Pausa a contagem, preservando o valor atual do TMR.
- **Bit 1 (TS - Timer Start):** Usado no modo One-shot. Escrever 1 e, em seguida, 0 reinicia o timer. É um pulso de disparo para iniciar uma nova contagem no modo One-shot.
- **Bit 2 (POE - PWM0 Output Enable):** 1: Habilita a saída no pino `pwm0`. 0: Desabilita a saída, forçando `pwm0` a um estado inativo (geralmente baixo).
- **Bit 3 (P1E - PWM1 Output Enable):** 1: Habilita a saída no pino `pwm1`. 0: Desabilita a saída, forçando `pwm1` a um estado inativo.

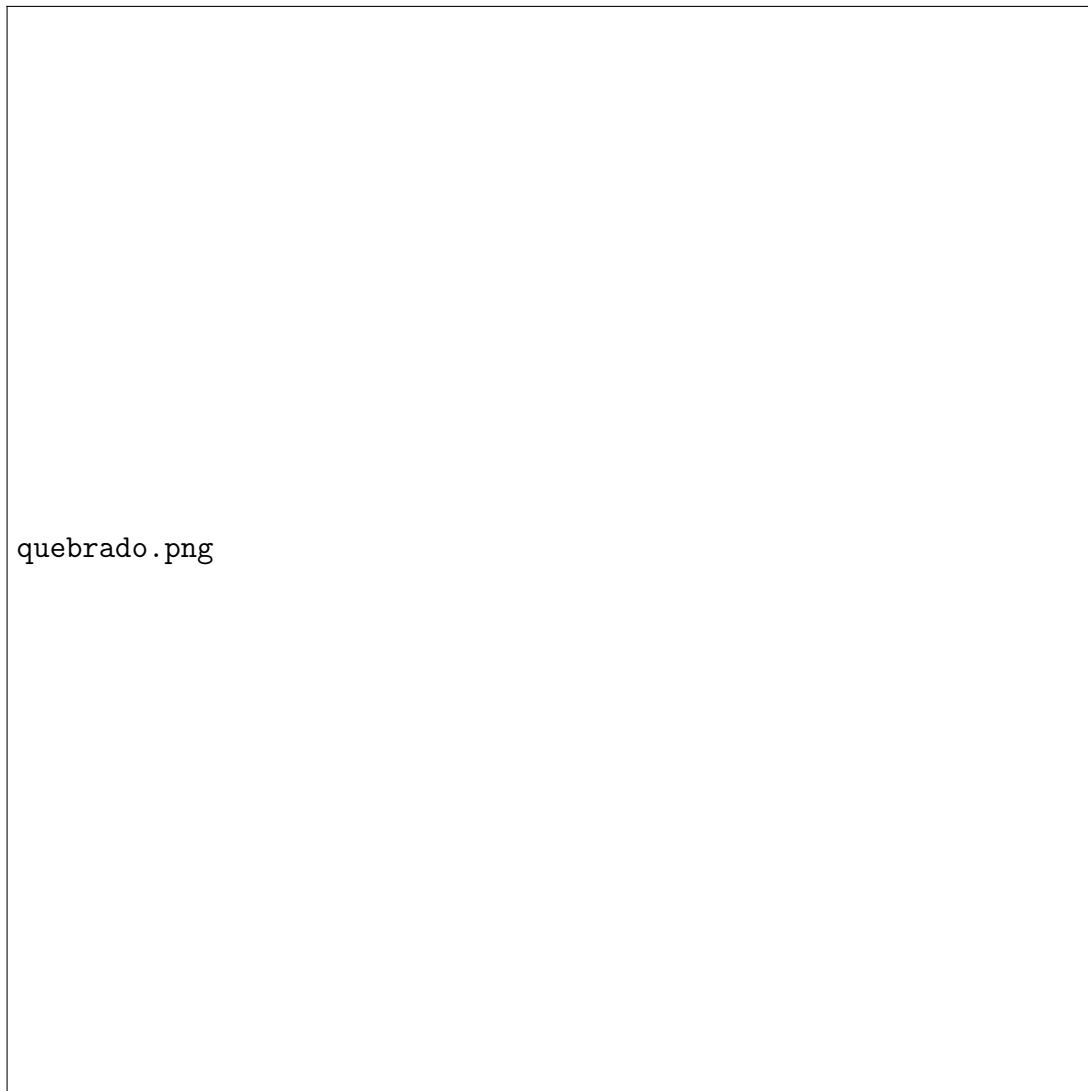


Figura 4.1: Diagramas de Mapeamento de Bits para TMR, RELOAD, PR, CMPX, CMPY e início de CTRL [1].

- **Bit 4 (DTE - Dead Time Enable):** 1: Habilita o gerador de *dead time* para os sinais PWM. 0: Desabilita o *dead time*.
- **Bit 5 (PI0 - PWM0 Invert):** 1: Inverte logicamente o sinal final de `pwm0`. 0: Saída `pwm0` não invertida.
- **Bit 6 (PI1 - PWM1 Invert):** 1: Inverte logicamente o sinal final de `pwm1`. 0: Saída `pwm1` não invertida.

4.2.3 Registrador CFG (0x0018)

Este registrador de 3 bits define o modo de contagem e operação do timer [2]. A Figura 4.2 detalha seu mapeamento de bits.

- **Bits 1:0 (DIR - Direction):** Define a direção de contagem do timer.
 - 10 (Up): O contador incrementa de 0 até RELOAD.
 - 01 (Down): O contador decrementa de RELOAD até 0.
 - 11 (Up-Down): O contador alterna entre incrementar (0 a RELOAD) e decrementar (RELOAD a 0).
- **Bit 2 (P - Periodic):** Define o comportamento do timer ao atingir o limite.
 - 1 (Periódico): O timer reinicia automaticamente após atingir o limite (RELOAD ou 0).
 - 0 (One-shot): O timer para após completar um ciclo (atingir RELOAD ou 0) e desabilita o bit TE.

4.2.4 Registradores PWM0CFG (0x1C) e PWM1CFG (0x20)

Estes registradores de 12 bits são a chave para a geração de formas de onda PWM complexas. Eles definem a ação da saída PWM em seis eventos distintos [3]. Cada evento é controlado por 2 bits, com a seguinte codificação:

- 00: **No Action** (Mantém o estado atual da saída).
- 01: **Low** (Força a saída para nível lógico baixo).
- 10: **High** (Força a saída para nível lógico alto).
- 11: **Invert** (Inverte o estado atual da saída).

Os campos de bits e seus eventos associados (verificados via RTL [3]):

Campo	Bits	Evento Associado
E0	1:0	Ação quando o Timer atinge zero (<code>tmr_eq_zero</code>).
E1	3:2	Ação quando o Timer atinge CMPX durante a contagem ascendente (<code>tmr_eq_cmpx</code> e <code>tmr_dir=1</code>).
E2	5:4	Ação quando o Timer atinge CMPY durante a contagem ascendente (<code>tmr_eq_cmpy</code> e <code>tmr_dir=1</code>).
E3	7:6	Ação quando o Timer atinge RELOAD (<code>tmr_eq_reload</code>).

Campo	Bits	Evento Associado
E4	9:8	Ação quando o Timer atinge CMPY durante a contagem descendente (<code>tmr_eq_cmpy</code> e <code>tmr_dir=0</code>).
E5	11:10	Ação quando o Timer atinge CMPX durante a contagem descendente (<code>tmr_eq_cmpx</code> e <code>tmr_dir=0</code>).



Figura 4.2: Diagramas de Mapeamento de Bits para CTRL, CFG e início de PWM0CFG [1].

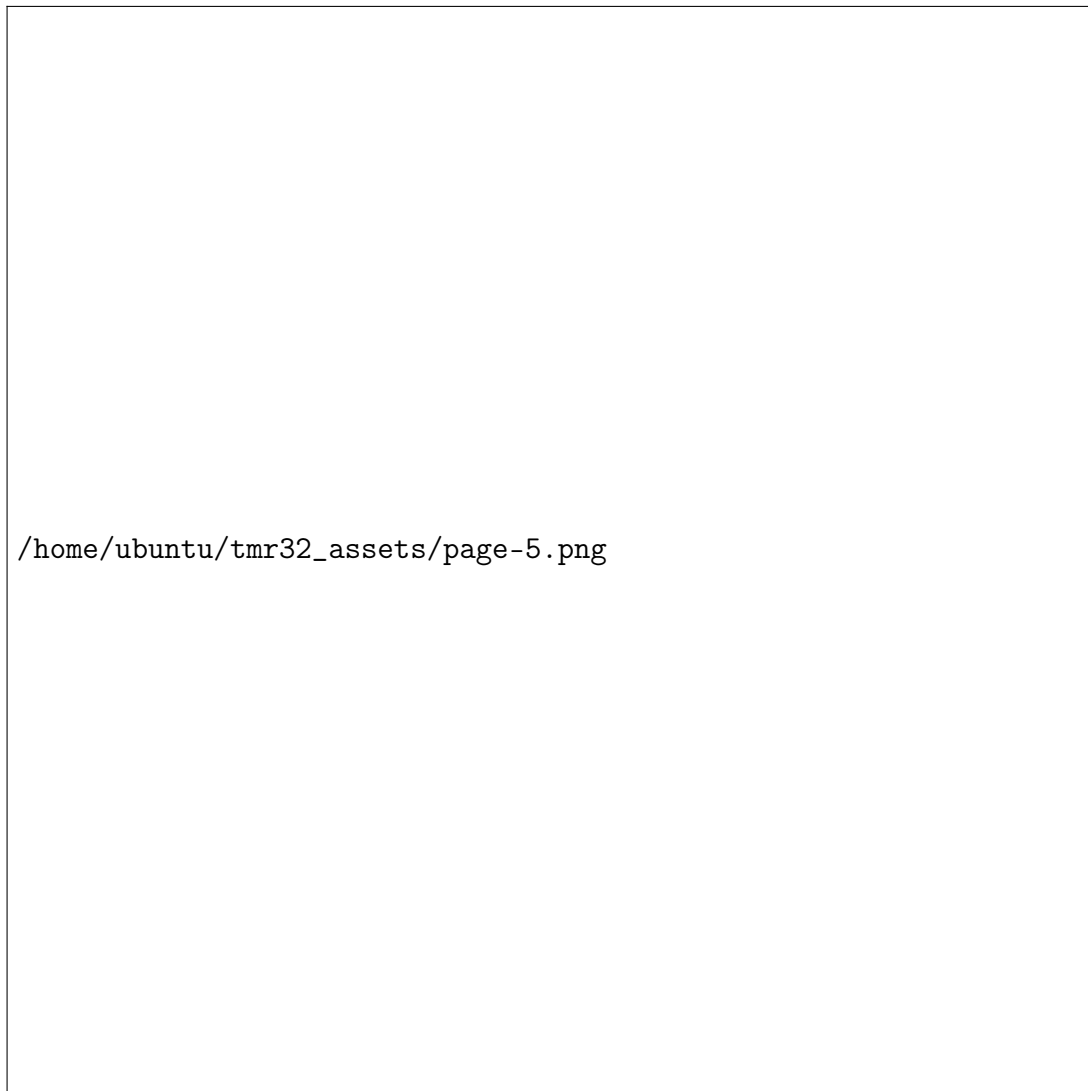


Figura 4.3: Diagramas de Mapeamento de Bits para PWM0CFG (continuação), PWM1CFG, PWMDT, PWMFC e GCLK [1].

4.2.5 Registrador PWMDT (0x0024)

Este registrador de 8 bits (`pwm_dt`) define o período de *dead time* em ciclos de *tick* (saída do prescaler) [2]. Quando DTE (CTRL[4]) está habilitado, as transições das saídas PWM são atrasadas por este valor, garantindo que um sinal se desative completamente antes que o outro se ative. O RTL revela que o `pwm1` atua como o complemento do `pwm0` com a banda morta inserida, o que é ideal para controle de meia-ponte.

4.2.6 Registrador PWMFC (0x0028)

Este registrador de 16 bits é usado para limpar a condição de falha do PWM. A limpeza não é um simples "escrever 1", mas uma **sequência de escrita específica** [3]:

1. Escrever o valor 16'hA539 (PWM_FAULT_CLR_C0) para ativar um sinal interno de pré-limpeza.
2. Em seguida, escrever o valor 16'hA953 (PWM_FAULT_CLR_C1) para efetivamente limpar a flag de falha e reabilitar as saídas PWM.

A Figura 4.3 detalha seu mapeamento de bits.

4.2.7 Registradores de Interrupção (IM, RIS, MIS, IC)

O sistema de interrupções opera em três estágios, conforme verificado no RTL [3]. A Figura 4.4 e a Tabela 4.3 detalham os flags de interrupção.

- **RIS (Raw Interrupt Status - 0xFF08) [R]**: Registrador de 3 bits que indica os eventos de interrupção brutos que ocorreram (Bit 0: Timeout, Bit 1: Match X, Bit 2: Match Y). É atualizado pelo hardware e é somente leitura.
- **IM (Interrupt Mask - 0xFF00) [RW]**: Registrador de 3 bits que mascara os eventos de interrupção. Um bit 1 permite que o evento correspondente no RIS contribua para o sinal IRQ; 0 o mascara.
- **MIS (Masked Interrupt Status - 0xFF04) [R]**: Registrador de 3 bits que reflete o status das interrupções após a aplicação da máscara ($MIS = RIS \text{ AND } IM$). Se qualquer bit aqui for 1, o pino IRQ externo é ativado.
- **IC (Interrupt Clear - 0xFF0C) [W]**: Registrador de 3 bits para limpar as interrupções. Escrever 1 em um bit limpa o bit correspondente no RIS. O registrador é auto-limpante após a escrita.

Tabela 4.3: Flags de Interrupção [1].

Bit	Flag	Width	Descrição
0	TO	1	Timeout: TMR matches 0 (down counting) or RELOAD (up counting).
1	MX	1	TMR matches CMPX register.
2	MY	1	TMR matches CMPY register.

4.2.8 Registrador GCLK (0xFF10)

Este registrador de 1 bit controla o *Clock Gating* do núcleo do timer [3]. A Figura 4.4 detalha seu mapeamento de bits.

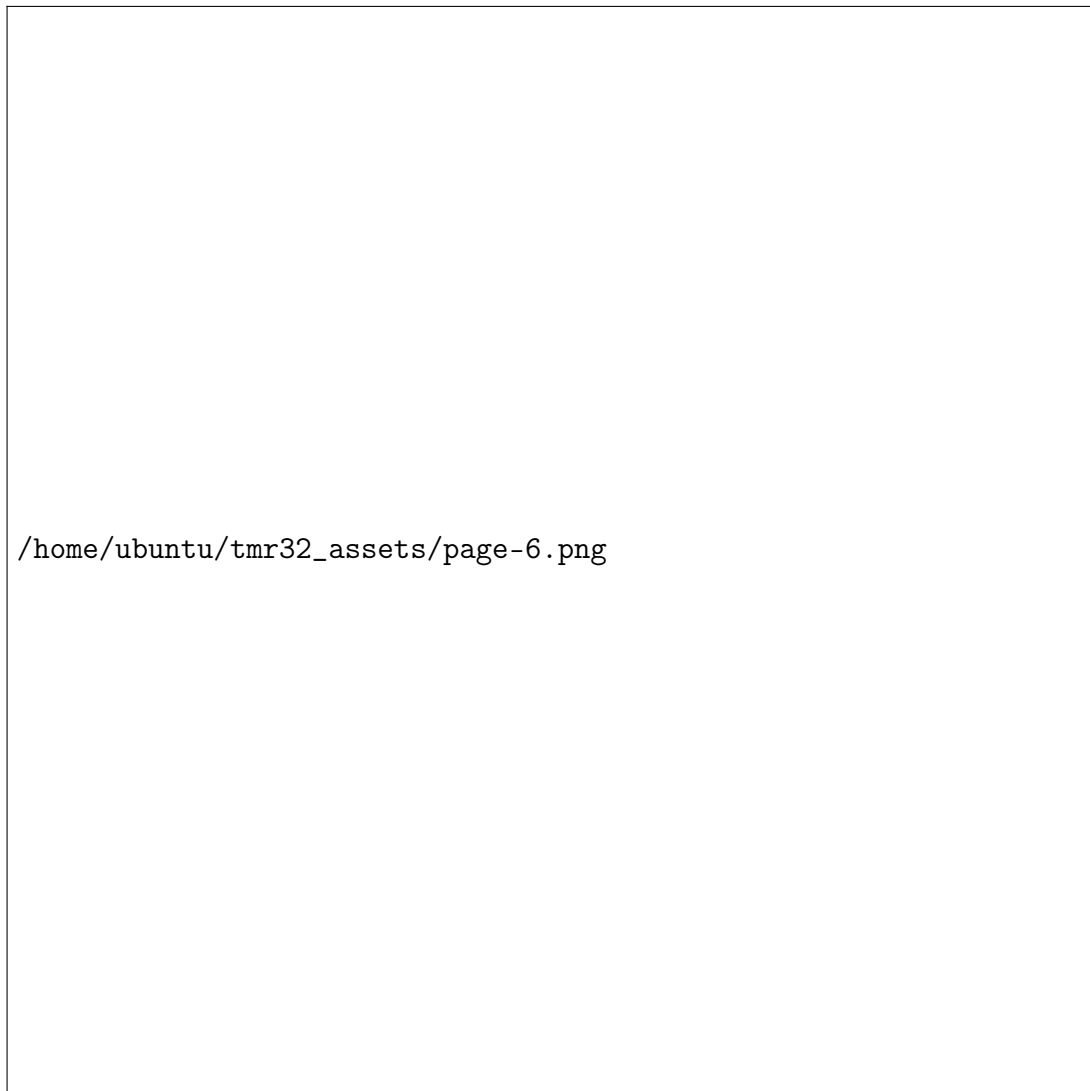


Figura 4.4: Diagrama de Mapeamento de Bits para GCLK e Flags de Interrupção [1].

- **Bit 0 (gclk_enable):** 1: Habilita o clock para o núcleo do timer. 0: Desabilita o clock para o núcleo, reduzindo o consumo de energia. O acesso aos registradores via Wishbone permanece funcional.

5 Geração de PWM Baseada em Eventos: Detalhes da Lógica

A lógica de geração de PWM do EF_TMR32 é altamente flexível devido à sua abordagem baseada em eventos. Em vez de um simples comparador, o IP utiliza uma **Máquina de Estados de Saída** para cada canal PWM. Cada transição da saída é determinada por uma combinação do estado atual do contador e das configurações nos registradores PWMxCFG.

Por exemplo, para criar um PWM básico com duty cycle de 50% no modo Up-Down, poderíamos configurar:

- RELOAD = 100 (período total de 200 ticks)
- CMPX = 50
- PWM0CFG: E0 (Timer=0) = High, E1 (Timer=CM PX Up) = Low, E3 (Timer=RELOAD) = Invert, E5 (Timer=CM PX Down) = Invert.

Isso faria com que a saída `pwm0` ficasse alta de 0 a 50 (subindo), baixa de 50 a 100 (subindo), invertesse em 100, ficasse alta de 100 a 50 (descendo), invertesse em 50, e ficasse baixa de 50 a 0 (descendo), invertendo em 0. O resultado seria um sinal com 50% de duty cycle centrado.

6 Interface de Controle Wishbone B4: Protocolo e Integração

O barramento Wishbone atua como o sistema nervoso do IP, permitindo que um mestre (CPU, DMA) configure e monitore o timer. Ele é implementado como um **Escravo Síncrono** [4].

6.1 Protocolo de Acesso Detalhado

Uma transação típica de leitura ou escrita segue este fluxo de *handshake* [4]:

1. **Início da Transação:** O Mestre levanta `cyc_i` e `stb_i`, colocando o endereço (`adr_i`) e os dados (`dat_i`, se for escrita) no barramento. O sinal `we_i` indica a direção da operação.
2. **Decodificação e Preparação:** O decodificador interno do EF_TMR32 reconhece o offset do registrador e prepara a leitura (colocando o valor do registrador em `dat_o`) ou a escrita (gravando `dat_i` no registrador alvo).
3. **Reconhecimento:** No ciclo de clock seguinte, o IP levanta `ack_o`, sinalizando ao mestre que a operação foi concluída com sucesso e que os dados de leitura (se aplicável) são válidos.
4. **Finalização:** O Mestre vê o `ack_o` e, no próximo ciclo, baixa os sinais de controle (`cyc_i`, `stb_i`), finalizando o ciclo de barramento.

Este protocolo robusto garante que não haja colisões de dados e que o processador possa operar em frequências diferentes da lógica interna do timer, se necessário, desde que a sincronização do barramento seja mantida.

7 Guia de Programação (Exemplos Práticos)

Exemplo 1: Timer Periódico de 1ms (Clock de 10MHz)

Assumindo um clock de sistema de 10MHz, queremos um período de 1ms. Isso significa 10.000 ciclos de clock. Para um timer de 32 bits, podemos usar o prescaler para ajustar a granularidade.

- **Cálculo do Prescaler:** Se $F_{clk_g} = 10MHz$, e queremos um *tick* a cada 1us (1MHz), então $PR + 1 = 10$, logo $PR = 9$.
- **Cálculo do RELOAD:** Para 1ms (1000us), precisamos de 1000 *ticks*. Então, $RELOAD = 999$ (contagem de 0 a 999 = 1000 ticks).
- **Configuração dos Registradores (Pseudocódigo):**

```
WRITE(GCLK, 0x1);           // Habilita Clock Gating
WRITE(PR, 9);               // Prescaler para 1us/tick
WRITE(RELOAD, 999);         // Período de 1ms
WRITE(CFG, 0x6);            // Modo Up (10), Periódico (1)
WRITE(CTRL, 0x1);           // Habilita Timer (TE=1)
```

Exemplo 2: PWM com 25% de Duty Cycle (Canal 0)

Para um PWM simples com 25% de duty cycle, podemos usar o modo Up. O período total será definido por RELOAD, e o ponto de transição por CMPX.

- **RELOAD:** 100 (Período de 101 ticks).
- **CMPX:** 25 (A saída muda em 25 ticks).
- **PWM0CFG:** Configurar E0 para High (10) e E1 para Low (01).
- **Configuração dos Registradores (Pseudocódigo):**

```
WRITE(RELOAD, 100);           // Período base
WRITE(CMPX, 25);              // Ponto de transição
// E0=High (10), E1=Low (01), outros No Action (00)
WRITE(PWM0CFG, (0b000000000110)); // (E5-E2=00, E1=01, E0=10)
WRITE(CFG, 0x4);              // Modo Up (10), One-shot (0) ou Periódico (1)
WRITE(CTRL, 0x5);             // Habilita Timer (TE=1), Habilita PWM0 (PWME=1)
```

8 Conclusão e Melhores Práticas

O EF_TMR32 não é apenas um contador, mas um subsistema de controle de sinais. Sua arquitetura baseada em eventos através dos registradores PWMxCFG permite a implementação de protocolos de comunicação simples, controle de brilho LED, e regulação complexa de motores, tudo com carga mínima sobre a CPU principal.

Para extrair o máximo do EF_TMR32, recomenda-se:

- Sempre configurar os registradores de limite (RELOAD, CMPX, CMPY) **antes** de habilitar o timer no CTRL. Isso evita comportamentos indefinidos durante a transição.
- Utilizar o *Clock Gating* (GCLK) agressivamente em dispositivos alimentados por bateria para maximizar a eficiência energética.
- Em sistemas de alta confiabilidade, conectar o sinal `pwm_fault` a sensores térmicos ou de sobrecorrente externos para proteção instantânea do hardware. A sequência de limpeza de falha deve ser cuidadosamente implementada no firmware.
- Para aplicações de controle de motores, o modo Up-Down com *dead time* é a configuração ideal para minimizar distorções harmônicas e proteger os drivers de potência.

Referências

- [1] Efabless. *EF_TMR32 Datasheet*. Disponível em: `/home/ubuntu/projects/jjkj-40baff52/EF_TMR32.pdf`
- [2] Efabless. *EF_TMR32 GitHub Repository*. Disponível em: https://github.com/efabless/EF_TMR32
- [3] Universidade Federal de Campina Grande - VIRTUS CC. *Wishbone RISC-X*. Disponível em: https://github.com/efabless/EF_TMR32/blob/main/hdl/rtl/EF_TMR32.v
- [4] OpenCores. *Wishbone Bus Architecture Specification B4*. Disponível em: `/home/ubuntu/projects/jjkj-40baff52/Wishbone(1).pdf`